MSc Large Databases

Module outline: from a single query to systems that scale

M Usman · Teaching Assistant, University of Huddersfield

Indicative module outline informed by canonical MSc curricula.

1 · Aims

How databases stay correct and fast as the data grows

M Usman · From enterprise data to production AI.

What students can do by the end

- Describe data at scale through the four Vs, and pick a data model that fits the workload
- Design a schema, reason about normalisation, and write analytical SQL
- Read a query plan and explain why one query runs and another scales
- Reason about transactions, concurrency, and recovery under failure
- Choose between relational, document, graph and warehouse architectures on evidence

36 hours

twelve one-hour lectures and twelve two-hour labs, paired one to one

Lectures 1–4 · the relational foundation

1. **Big data and the database families** · the four Vs (volume, velocity, variety, veracity); OLTP against OLAP; the five data models (relational, key-value, wide-column, document, graph) and the problems each suits
2. **The relational model and schema design** · relations, tuples, keys, relational algebra; entity–relationship modelling and the mapping to relations; entity, referential and domain integrity
3. **Normalisation** · functional dependencies, Armstrong's axioms, attribute closure; 1NF to BCNF and the anomalies each removes; lossless-join decomposition; 4NF and pragmatic denormalisation
4. **SQL, from queries to analytics** · joins, subqueries, aggregation; window functions, recursive common table expressions, ROLLUP and CUBE; the three-valued logic of NULL

Lectures 5–8 · inside the engine

5. **Relational engines: PostgreSQL and MySQL** · the client and server model, the system catalog, multi-version concurrency; PostgreSQL heap storage and VACUUM against MySQL InnoDB's clustered primary key
6. **Storage and indexing** · the memory hierarchy, pages and the buffer pool, slotted pages; heap against clustered storage; B+-trees, hash indexes, and the LSM-tree for write-heavy stores
7. **Query processing and optimisation** · the path from logical to physical plan; nested-loop, hash and sort-merge joins; cost-based optimisation, selectivity and statistics; reading EXPLAIN ANALYZE
8. **Transactions and concurrency** · ACID, serialisability and the standard anomalies; the SQL isolation levels; two-phase locking, deadlock handling, and snapshot isolation

Lectures 9–12 · reliability, NoSQL and scale

9. **Recovery and durability** · failure models; the write-ahead log and the ARIES algorithm (analysis, redo, undo); checkpoints, force and steal policies, log sequence numbers
10. **NoSQL and document databases** · schema-on-read, horizontal scale, the BASE properties; the four NoSQL families; MongoDB documents, embedding against referencing, the aggregation pipeline, replica sets
11. **Graph databases** · the property-graph model and index-free adjacency; Neo4j and Cypher (pattern matching, variable-length paths, shortest path); key-value (Redis) and wide-column (HBase) for contrast
12. **Distribution, scale and the warehouse** · partitioning and replication; the CAP theorem and its PACELC refinement; star and snowflake schemas, OLAP, and columnar storage

Labs 1–4 · schema, SQL and analytics

1. **Load at scale (PostgreSQL)** · ingest a real public dataset with COPY; measure load time, on-disk size and row counts to make the four Vs concrete
2. **ER modelling to schema (PostgreSQL)** · turn a domain brief into an ER diagram, then DDL with keys and constraints; validate with valid and constraint-violating inserts
3. **Normalisation to BCNF (PostgreSQL)** · from a denormalised table, identify functional dependencies, decompose to 3NF then BCNF, and prove the decomposition lossless by reconstruction
4. **Advanced SQL analytics (PostgreSQL)** · window functions, recursive CTEs, ROLLUP and CUBE; compare correctness and clarity against the equivalent subquery formulations

Labs 5–8 · performance and correctness

5. **Indexing and measured query time (PostgreSQL · MySQL)** · baseline a query, add B+-tree and partial indexes, measure latency and index size before and after; contrast clustered against heap on InnoDB
6. **Query plans and optimisation (PostgreSQL)** · read EXPLAIN ANALYZE for multi-join queries, observe join-order changes, refresh statistics, and see how cardinality estimates determine the physical plan
7. **Isolation anomalies (PostgreSQL)** · with two sessions, reproduce dirty read, non-repeatable read, phantom and lost update, then re-run at higher isolation levels and record what disappears and at what cost
8. **WAL and crash recovery (PostgreSQL)** · inspect the write-ahead log, force a checkpoint, simulate a crash, and watch redo and undo restore a correct, durable state on restart

Labs 9–12 · NoSQL, polyglot and a capstone

9. **Document modelling (MongoDB)** · model the Lab 2 domain as documents, weigh embedding against referencing, build an aggregation pipeline, and measure a secondary index
10. **Graph traversal (Neo4j)** · load a connected dataset, write Cypher pattern-match, variable-length-path and shortest-path queries, and contrast with a recursive join in PostgreSQL
11. **Polyglot persistence (PostgreSQL · MongoDB · Neo4j)** · solve one workload across relational, document and graph stores; record modelling effort, expressiveness and latency, then recommend a fit
12. **Capstone** · partition and replicate a large table, observe replication lag and read scaling, then build a star-schema warehouse and run OLAP roll-up and drill-down

Tools and datasets

- **PostgreSQL** and **MySQL** for relational work; **MongoDB** for documents and **Neo4j** for graphs in the NoSQL units
- Query-plan inspection from the outset, reading what the optimiser actually does rather than what you assume
- Production-scale public datasets, so indexing and optimisation decisions have visible consequences

How it is taught

- Each lecture connects a layer of the stack to the failure it prevents or the workload it serves
- Each two-hour lab is hands-on: a measured experiment on a live engine, not a paper exercise
- The recurring question: what happens to this design when the data is a hundred times larger?

Why it matters

Every analytics and AI system rests on the data layer. Knowing the internals is the difference between a query that runs and one that scales.

In summary

Core texts: Silberschatz, Korth & Sudarshan, Database System Concepts (7th ed.) · Ramakrishnan & Gehrke, Database Management Systems (3rd ed.) · Kleppmann, Designing Data-Intensive Applications · Perkins, Redmond & Wilson, Seven Databases in Seven Weeks (2nd ed.)

M Usman · Teaching Assistant, University of Huddersfield · mtusman-ai.github.io/M.Usman

See the full portfolio

The work behind this deck sits on one page: the Insights series, the projects and the code that runs them, and the publications.

mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.